Mathematical Foundations, Architecture Diagrams, and State Analysis of Modern LLM Training Methods — Part 2: Preference Alignment, Reinforcement Learning, and Reward Modeling

Chief Strategist J

June 12, 2026

Contents

Part I
Preference Alignment Methods

Reinforcement Learning from Human Feedback (RLHF)

Mathematical Foundation

RLHF aligns language models with human preferences in a two-stage process.

First, a Reward Model (RM) $r_\phi(x, y)$ is trained on a dataset of human comparisons $\mathcal{D} = \{(x, y_w, y_l)\}$, where y_w is preferred to y_l given prompt x. The preference is modeled using the Bradley-Terry preference model:

$$P(y_w \succ y_l \mid x) = \sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l)\big) = \frac{1}{1 + \exp\big(-(r_\phi(x, y_w) - r_\phi(x, y_l))\big)} \tag{1}$$

The RM is trained by minimizing the negative log-likelihood of this pairwise choice:

$$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E}_{(x, y_w, y_l)\sim\mathcal{D}}\left[\log \sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l)\big)\right] \tag{2}$$

Second, the SFT policy parameters θ are fine-tuned to maximize the expected reward under the learned RM, while regularized by a Kullback-Leibler (KL) divergence constraint to the reference policy π_{ref} (preventing policy collapse and reward hacking):

$$\mathcal{J}_{\text{RLHF}}(\theta) = \mathbb{E}_{x\sim\mathcal{D}_x, y\sim\pi_\theta(\cdot|x)}\left[r_\phi(x, y) - \beta D_{\text{KL}}\big(\pi_\theta(y \mid x) \,\|\, \pi_{\text{ref}}(y \mid x)\big)\right] \tag{3}$$

where β is a regularizing coefficient, and the KL term at each token generation step is factorized as:

$$D_{\text{KL}}\big(\pi_\theta(y \mid x) \,\|\, \pi_{\text{ref}}(y \mid x)\big) = \sum_{t=1}^{T} \log \frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\text{ref}}(y_t \mid x, y_{<t})} \tag{4}$$

Architecture Flow Diagram

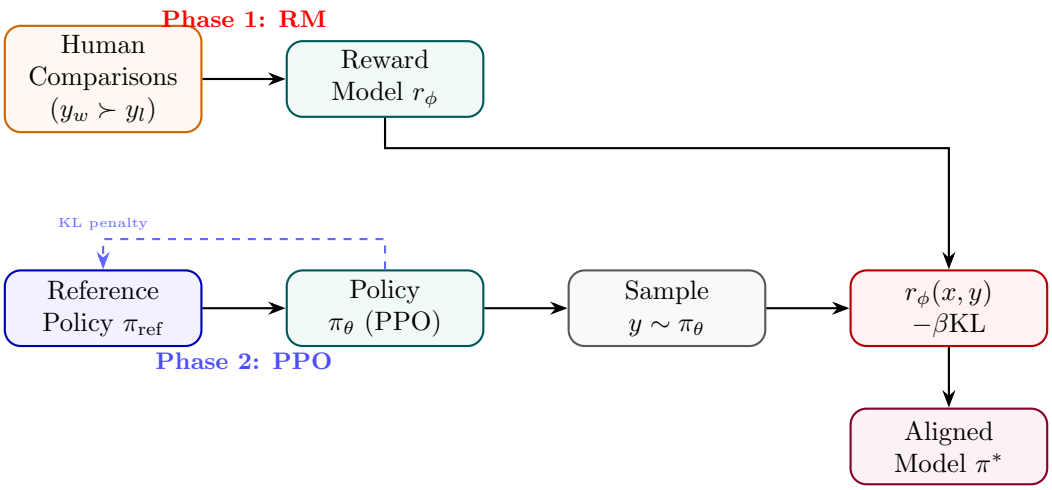

Figure 1: RLHF: reward model trained on human comparisons then used to guide PPO policy optimisation.

State Transition Table

Property	Before	After
Policy type	SFT checkpoint	RLHF-aligned policy
Reward signal	None	Human preference score
KL from reference	0	Bounded by β
Human preference win-rate	$\sim$50%	$> 70\%$
Training complexity	Single loop	Two-phase (RM + PPO)

Table 1: State properties before and after RLHF.

Direct Preference Optimization (DPO)

Mathematical Foundation

DPO eliminates the need for training an explicit reward model or running reinforcement learning. Starting from the KL-constrained RL objective:

$$\max_{\pi} \mathbb{E}_{x,y\sim\pi}\left[r(x,y)\right] - \beta D_{\mathrm{KL}}(\pi(y \mid x) \,\|\, \pi_{\mathrm{ref}}(y \mid x)) \tag{5}$$

we formulate the unconstrained objective using a partition function $Z(x)$:

$$\max_{\pi} \mathbb{E}_{x}\left[\mathbb{E}_{y\sim\pi}\left[r(x,y) - \beta \log \frac{\pi(y \mid x)}{\pi_{\mathrm{ref}}(y \mid x)}\right]\right] \tag{6}$$

The closed-form optimal policy solution $\pi^*(y \mid x)$ under this objective is:

$$\pi^*(y \mid x) = \frac{1}{Z(x)}\pi_{\mathrm{ref}}(y \mid x)\exp\left(\frac{r(x,y)}{\beta}\right), \quad Z(x) = \sum_{y} \pi_{\mathrm{ref}}(y \mid x)\exp\left(\frac{r(x,y)}{\beta}\right) \tag{7}$$

By rearranging this relation, we express the implicit reward function $r(x,y)$ in terms of the policy likelihoods:

$$r(x,y) = \beta \log \frac{\pi^*(y \mid x)}{\pi_{\mathrm{ref}}(y \mid x)} + \beta \log Z(x) \tag{8}$$

Substituting this expression into the Bradley-Terry preference probability $P(y_w \succ y_l \mid x) = \sigma(r(x,y_w) - r(x,y_l))$, the partition function $Z(x)$ cancels out, resulting in the DPO loss function:

$$\mathcal{L}_{\mathrm{DPO}}(\theta;\pi_{\mathrm{ref}}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\mathrm{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\mathrm{ref}}(y_l \mid x)}\right)\right] \tag{9}$$

Architecture Flow Diagram

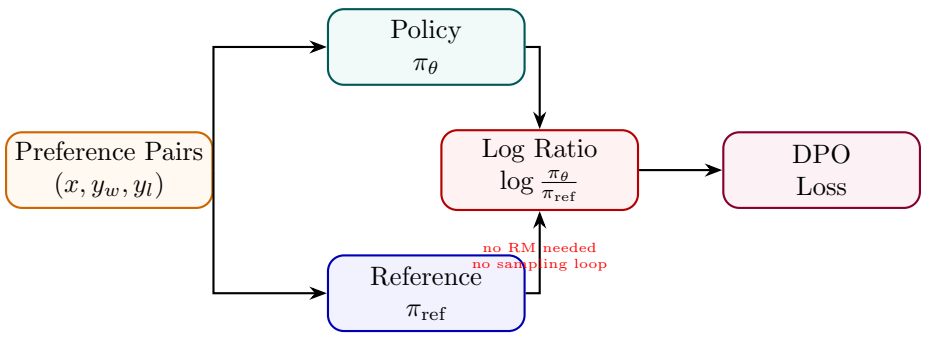

Figure 2: DPO: both policy and frozen reference score chosen/rejected responses; loss uses their log-ratio difference.

State Transition Table

Property	Before (RLHF)	After (DPO)
Reward model	Explicit r_ϕ	Implicit in log-ratio
Sampling loop	Yes (PPO rollouts)	No
Reference policy	Frozen copy	Frozen copy
Training stability	Sensitive to reward hacking	More stable
Compute cost	High (2-phase)	Lower (single SFT-like pass)

Table 2: State properties comparing RLHF and DPO.

Identity Preference Optimization (IPO)

Mathematical Foundation

IPO relaxes the Bradley-Terry preference assumption of DPO to prevent overfitting and improve generalization on preference pairs. By applying identity mapping directly to the pairwise probability, the IPO objective minimizes the mean squared error between the policy log-ratio difference and a target preference margin:

$$\mathcal{L}_{\text{IPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta\log\frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right] + \text{regularization} \tag{10}$$

Formally, IPO minimizes the quadratic loss of the difference of log-ratios relative to a regularizing parameter τ:

$$\mathcal{L}_{\text{IPO}}(\theta) = \mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\left(\log\frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \log\frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} - \frac{\tau}{2\beta}\right)^2\right] \tag{11}$$

This bounds the likelihood shift of both preferred and dispreferred sequences, preventing the model from outputting degenerate probabilities for preferred responses.

Architecture Flow Diagram

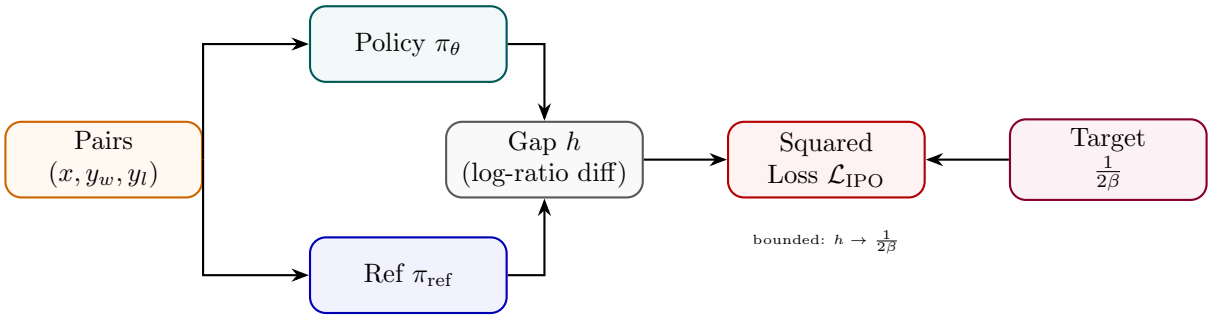

Figure 3: IPO: like DPO but the log-ratio gap is penalised toward a finite target $1/(2\beta)$.

State Transition Table

Property	Before (DPO)	After (IPO)
Loss type	Log-sigmoid	Squared error
Divergence at optimum	Unbounded	Bounded by $\frac{1}{2\beta}$
Target preference gap	Maximise	Converge to $\frac{1}{2\beta}$
Regularisation	KL implicit	Explicit squared bound
Training saturation	Can saturate	More stable

Table 3: State properties comparing DPO and IPO.

Odds Ratio Preference Optimization (ORPO)

Mathematical Foundation

ORPO combines SFT loss and an odds-ratio contrast in a single objective no reference model needed. The odds of generating y under policy π_θ is $\text{odds}(y|x) = \pi_\theta(y|x)/(1-\pi_\theta(y|x))$. The combined loss is:

$$\mathcal{L}_{\text{ORPO}} = \mathcal{L}_{\text{SFT}} - \lambda\,\mathbb{E}\left[\log\sigma\left(\log\frac{\text{odds}(y_w|x)}{\text{odds}(y_l|x)}\right)\right] \tag{12}$$

The SFT term increases $\pi_\theta(y_w|x)$ while the odds-ratio term simultaneously penalises y_l without requiring a frozen reference copy.

Architecture Flow Diagram

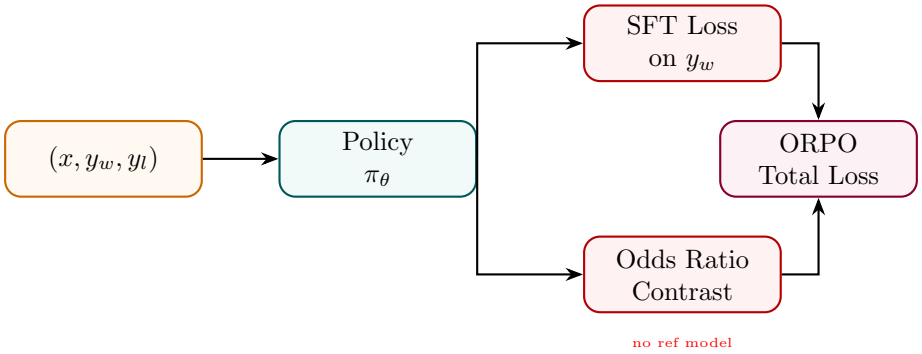

Figure 4: ORPO: SFT loss on the chosen response combined with an odds-ratio preference contrast.

State Transition Table

Property	**Before**	**After**
Reference model	Required (DPO/RLHF)	Not required
Training phases	2 (SFT then align)	1 (joint)
Loss terms	Single	SFT + odds-ratio
Memory overhead	2× model	1× model only
Data format	Prompt + completion	Triplet (x, y_w, y_l)

Table 4: State properties before and after ORPO.

Kahneman-Tversky Optimization (KTO)

Mathematical Foundation

KTO aligns language models using binary preference labels (e.g., correct/incorrect, helpful/harmful) instead of comparative pairs. This objective mathematically models human utility based on Kahneman and Tversky's prospect theory, which describes how humans weigh losses and gains asymmetricaly. Let y be a response to prompt x, and $r \in \{0, 1\}$ be its correctness label. The utility of the policy is defined as:

$$\mathcal{L}_{\text{KTO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{x,y}\left[w(y) \cdot v\left(\log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)} - z(x)\right)\right] \tag{13}$$

where $v(u)$ is the value function mapping gains and losses:

$$v(u) = \begin{cases} u & \text{if } u > 0 \\ \lambda u & \text{if } u \leq 0 \end{cases} \tag{14}$$

where $\lambda > 1$ represents loss aversion. The term $z(x)$ is a running reference point representing the expected log-ratio of the policy relative to the reference:

$$z(x) = \mathbb{E}_{y' \sim \pi_{\text{ref}}(\cdot|x)}\left[\log \frac{\pi_\theta(y' \mid x)}{\pi_{\text{ref}}(y' \mid x)}\right] \tag{15}$$

and $w(y)$ is a weighting factor balancing positive and negative feedback frequencies.

Architecture Flow Diagram

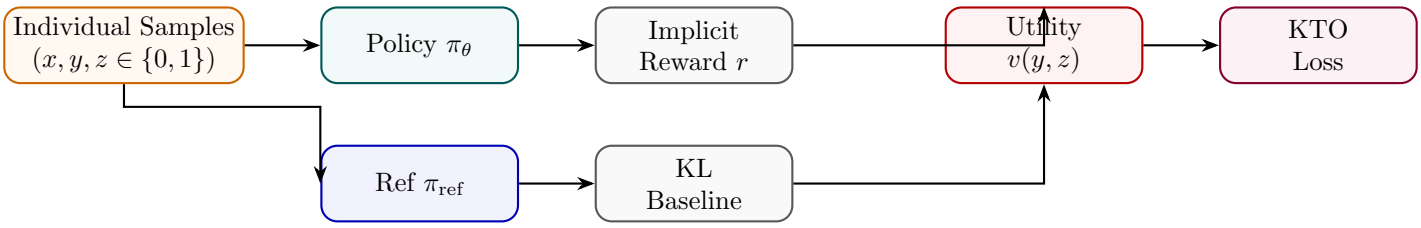

Figure 5: KTO: each sample is individually labelled good/bad; utility function uses prospect theory asymmetry.

State Transition Table

Property	**Before**	**After**
Data format	Pairwise (y_w, y_l)	Individual (y, z)
Data collection effort	High (pairwise)	Lower (binary label)
Loss-aversion modelling	None	Asymmetric w_+, w_-
Reference model	Optional	Used for KL baseline
Applicable at scale	Full pairs required	Works with any ratio

Table 5: State properties before and after KTO.

Simple Preference Optimization (SimPO)

Mathematical Foundation

SimPO improves upon DPO by removing the reference policy π_{ref} during training, reducing memory usage, and introducing a length-normalized reward with a target margin γ. The SimPO implicit reward for a sequence y given x is defined as the average log-likelihood:

$$r_{\text{SimPO}}(x, y) = \frac{\beta}{|y|} \log \pi_\theta(y \mid x) \tag{16}$$

where $|y|$ is the sequence length in tokens. The SimPO pairwise loss function is formulated as:

$$\mathcal{L}_{\text{SimPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}} \left[\log \sigma \left(\frac{\beta}{|y_w|} \log \pi_\theta(y_w \mid x) - \frac{\beta}{|y_l|} \log \pi_\theta(y_l \mid x) - \gamma \right) \right] \tag{17}$$

where $\gamma > 0$ is a target margin that forces the average likelihood of the winning response to exceed the losing response by a strict buffer, preventing over-optimization of short sequences.

Architecture Flow Diagram

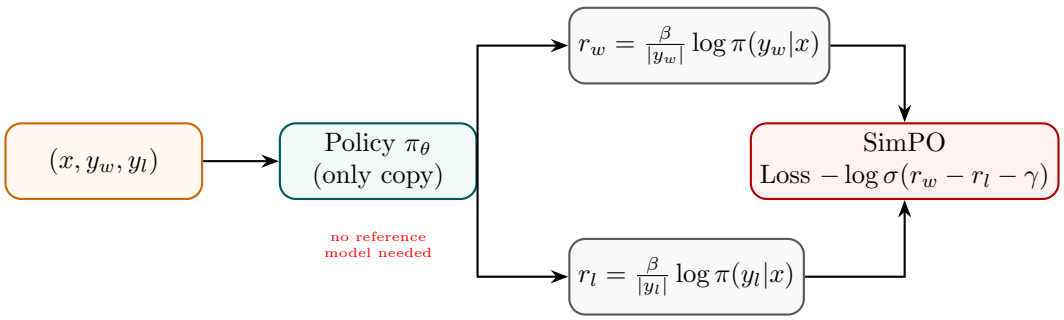

Figure 6: SimPO: length-normalised log-probs replace the reference log-ratio; margin γ enforces quality gap.

State Transition Table

Property	Before (DPO)	After (SimPO)
Reference model	Required	Not needed
Reward definition	Log-ratio π/π_{ref}	Length-norm avg log-prob
Length bias	Possible	Mitigated by $1/\|y\|$
Margin hyperparameter	None	$\gamma > 0$
Memory saving	None	No second model forward pass

Table 6: State properties comparing DPO and SimPO.

Constrained Preference Optimization (CPO)

Mathematical Foundation

CPO formulates alignment as a constrained optimisation: maximise reward while staying within a trust region ϵ of the reference policy measured in reverse KL divergence:

$$\max_{\pi_\theta} \mathbb{E}[r(x,y)] \quad \text{s.t.} \quad \mathbb{KL}[\pi_{\text{ref}} \| \pi_\theta] \leq \epsilon \tag{18}$$

The Lagrangian relaxation yields a combined training objective:

$$\mathcal{L}_{\text{CPO}} = -\mathbb{E}[r(x,y)] + \mu\big(\mathbb{KL}[\pi_{\text{ref}} \| \pi_\theta] - \epsilon\big) \tag{19}$$

where $\mu \geq 0$ is a dual variable updated online to enforce the constraint.

Architecture Flow Diagram

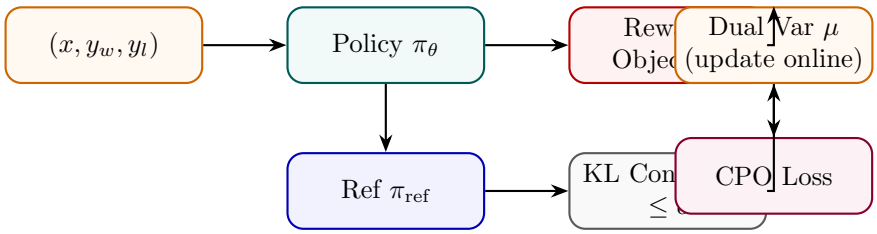

Figure 7: CPO: reward maximisation with an explicit KL constraint; dual variable μ adapts online.

State Transition Table

Property	**Before**	**After**
Constraint form	Soft KL penalty	Hard KL constraint $\leq \epsilon$
Dual variable μ	Fixed	Dynamically updated
Policy deviation	Unbounded	Bounded within trust region
Reward maximisation	Soft	Subject to constraint
Training dynamics	Standard	Saddle-point optimisation

Table 7: State properties before and after CPO.

Anchored Preference Optimization (APO)

Mathematical Foundation

APO introduces an anchor term that explicitly regularises the policy toward the reference model while simultaneously pushing chosen responses up and rejected responses down:

$$\mathcal{L}_{\text{APO}} = \mathbb{E}\left[\underbrace{-\log\sigma\big(\beta(r_w - r_l)\big)}_{\text{preference}} + \lambda\underbrace{\Big(\log\frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log\frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\Big)^2}_{\text{anchor}}\right] \tag{20}$$

The anchor term is minimised when both the chosen and rejected log-ratios equal a fixed reference, preventing the model from drifting.

Architecture Flow Diagram

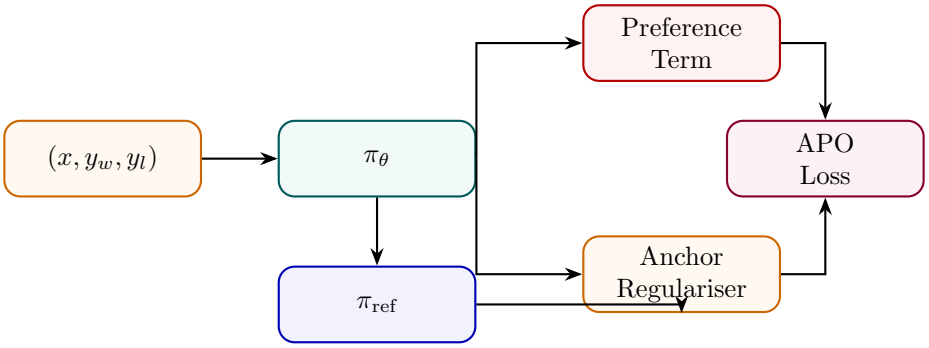

Figure 8: APO: combines standard preference loss with an anchor regulariser against the reference model.

State Transition Table

Property	Before (DPO)	After (APO)
Anchor regularisation	Implicit KL	Explicit squared anchor
Reference role	Log-ratio denominator	Explicit target
Policy drift	Can drift	Penalised
Loss terms	1	2 (preference + anchor)
Stability	Moderate	Higher (dual grounding)

Table 8: State properties comparing DPO and APO.

Rank Responses from Human Feedback (RRHF)

Mathematical Foundation

RRHF generates K candidate responses $\{y^{(1)}, \ldots, y^{(K)}\}$ from the model, ranks them by human or AI annotators, and trains the model to assign higher log-probability to higher-ranked responses via a pairwise ranking loss:

$$\mathcal{L}_{\text{RRHF}} = \sum_{i<j} \max\bigl(0,\, \text{score}(y^{(j)}) - \text{score}(y^{(i)}) + \delta\bigr) + \lambda \mathcal{L}_{\text{SFT}}(y^{(1)}) \tag{21}$$

where $\text{score}(y^{(k)}) = \log P_\theta(y^{(k)}|x)/|y^{(k)}|$ is the normalised log-prob. The SFT term anchors the top-ranked response.

Architecture Flow Diagram

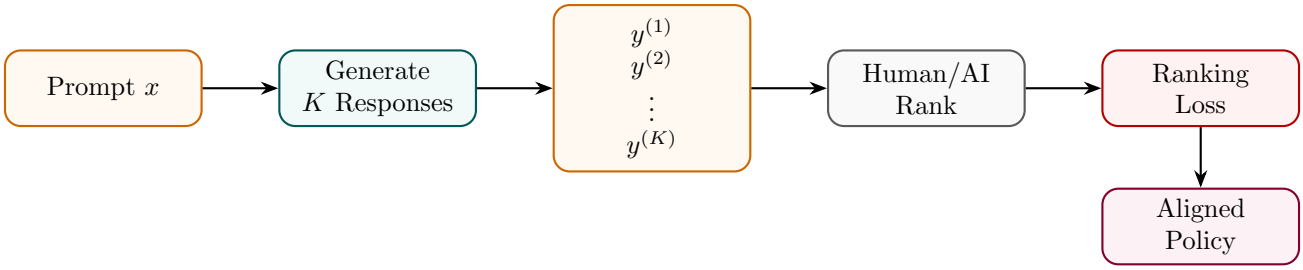

Figure 9: RRHF: K responses are generated and ranked; policy trained to match rank order via pairwise loss.

State Transition Table

Property	**Before**	**After**
Responses per prompt	1	K (ranked)
Supervision signal	Binary (win/lose)	Full rank ordering
Loss type	Log-sigmoid	Pairwise margin
Top-response training	Separate SFT	Joint SFT + ranking
Annotation granularity	Single comparison	Full K-way ranking

Table 9: State properties before and after RRHF.

Reinforcement Learning from AI Feedback (RLAIF)

Mathematical Foundation

RLAIF replaces expensive human annotators with a capable AI model ("AI annotator" $\mathcal{A}$) to generate preference labels at scale. The AI annotator scores pairs via a prompt p:

$$\hat{r}(x, y_i, y_j) = P_{\mathcal{A}}(y_i \succ y_j \mid p, x, y_i, y_j) \tag{22}$$

These AI-generated preferences are then used identically to human preferences to train the RM and run PPO (or DPO):

$$\mathcal{L}_{\text{RLAIF}} = -\mathbb{E}[\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))], \quad (y_w \succ y_l) \text{ from } \mathcal{A} \tag{23}$$

Architecture Flow Diagram

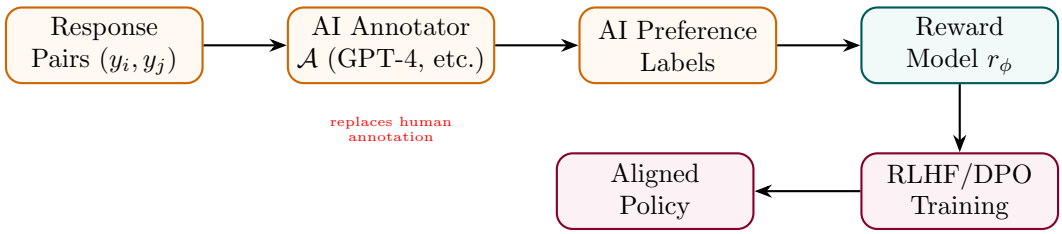

Figure 10: RLAIF: an AI model generates preference labels at scale, replacing costly human annotators.

State Transition Table

Property	Before (RLHF)	After (RLAIF)
Annotation source	Humans	AI annotator $\mathcal{A}$
Annotation cost	High ($$$)	Low (API calls)
Annotation scale	Limited (thousands)	Massive (millions)
Annotation consistency	Variable	Consistent per model
Annotation bias	Human bias	AI model bias

Table 10: State properties comparing RLHF and RLAIF.

Constitutional AI

Mathematical Foundation

Constitutional AI uses a set of principles $\mathcal{C} = \{c_1, \ldots, c_M\}$ (the "constitution") to iteratively critique and revise model outputs before training. In the RL phase, an AI Feedback model scores responses against the constitution:

$$r_{\text{CAI}}(x, y) = \frac{1}{M} \sum_{m=1}^{M} P_{\mathcal{A}}(\text{harmless} \mid c_m, x, y) \tag{24}$$

Two training phases: (1) supervised revision via critique-revise loop, (2) RLHF with AI-generated preference labels from the constitution.

Architecture Flow Diagram

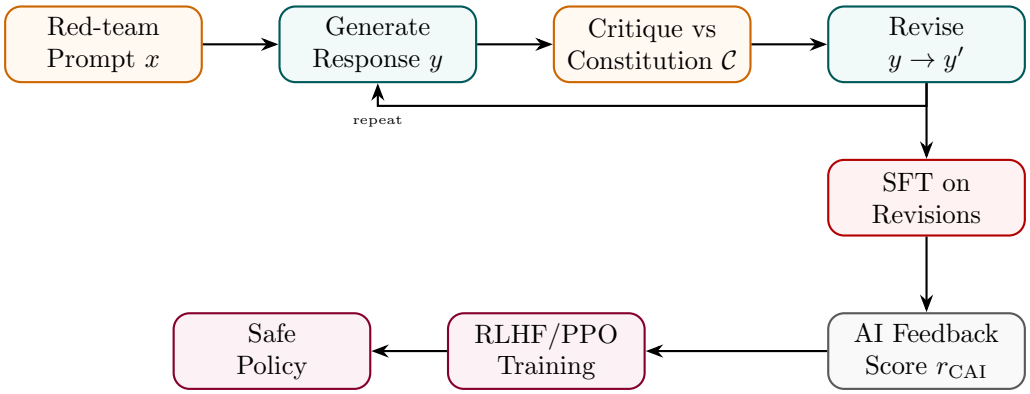

Figure 11: Constitutional AI: critique-revise loop creates SFT data; AI feedback from constitution guides RLHF.

State Transition Table

Property	Before	After
Safety signal	None / Human labels	AI + constitutionl principles
Training phases	SFT only	Critique-revise SFT + RLAIF
Red-team coverage	Manual	Systematic via constitution
Harmlessness	Moderate	Substantially improved
Helpfulness trade-off	N/A	Explicitly balanced

Table 11: State properties before and after Constitutional AI training.

Part II
Reinforcement Learning Training Methods

Proximal Policy Optimization (PPO)

Mathematical Foundation

PPO optimizes policy parameters θ using a clipped objective function to prevent destabilizing parameter updates. Let $\rho_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ be the action probability ratio. The surrogate objective maximizes:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(\rho_t(\theta)\hat{A}_t,\ \text{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right] \tag{25}$$

where ϵ is a clipping hyperparameter (typically 0.1 or 0.2), and $\hat{A}_t$ is the advantage estimated using Generalized Advantage Estimation (GAE):

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}^V, \quad \delta_t^V = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t) \tag{26}$$

where V_ϕ is the value network parameters, γ is the discount factor, and λ controls bias-variance trade-offs. The value function loss is clipped to match policy updates:

$$\mathcal{L}^{\text{VF}}(\phi) = \hat{\mathbb{E}}_t \left[\max \left((V_\phi(s_t) - V_t^{\text{target}})^2,\ (V_{\phi_{\text{old}}}(s_t) + \text{clip}(V_\phi(s_t) - V_{\phi_{\text{old}}}(s_t), -c, c) - V_t^{\text{target}})^2 \right) \right] \tag{27}$$

Architecture Flow Diagram

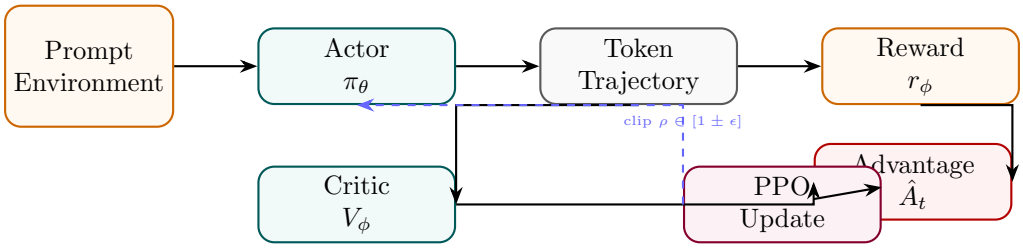

Figure 12: PPO: actor generates trajectory, critic estimates value, advantage drives clipped surrogate update.

State Transition Table

Property	Before	After
Policy update size	Unconstrained	Clipped by ϵ
Value network	None	Trained V_ϕ
Advantage estimation	N/A	GAE $\hat{A}_t$
Training stability	Unstable (vanilla PG)	Stable (clip constraint)
Reward signal	Sparse (end of seq)	Discounted returns

Table 12: State properties before and after PPO setup.

Group Relative Policy Optimization (GRPO)

Mathematical Foundation

GRPO optimizes policies by calculating rewards relative to a group of sampled outputs, eliminating the need for a separate critic model. For a given prompt x, the policy generates a group of G candidate completions $\{y_1, y_2, \ldots, y_G\}$. Each candidate y_i is evaluated by a reward function (or judge) yielding score r_i. The relative advantage A_i of each candidate is computed as:

$$A_i = \frac{r_i - \text{mean}(r)}{\text{std}(r)} = \frac{r_i - \frac{1}{G}\sum_j r_j}{\sqrt{\frac{1}{G}\sum_j (r_j - \bar{r})^2 + \epsilon}} \tag{28}$$

The policy objective maximizes the clipped advantage with a KL penalty directly evaluated on the generated tokens:

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\frac{1}{G}\sum_{i=1}^{G}\left[\min\left(\rho_i(\theta)A_i,\ \text{clip}(\rho_i(\theta), 1-\epsilon, 1+\epsilon)A_i\right) - \beta D_{\text{KL}}\left(\pi_\theta(y_i \mid x) \,\|\, \pi_{\text{ref}}(y_i \mid x)\right)\right] \tag{29}$$

where the probability ratio $\rho_i(\theta)$ is defined as:

$$\rho_i(\theta) = \frac{\pi_\theta(y_i \mid x)}{\pi_{\theta_{\text{old}}}(y_i \mid x)} \tag{30}$$

Architecture Flow Diagram

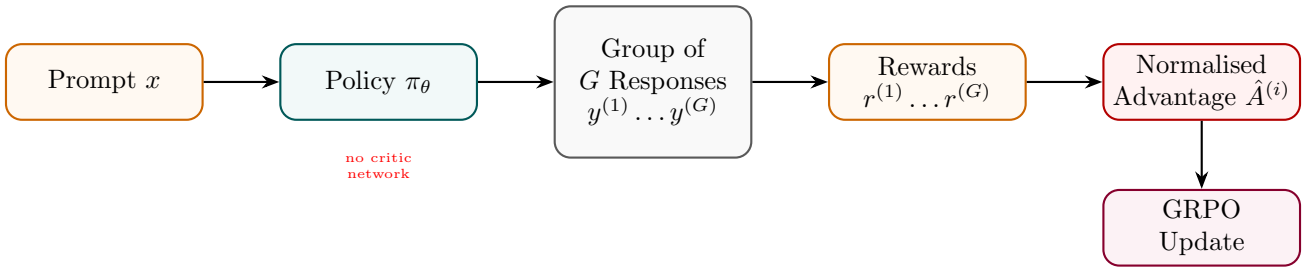

Figure 13: GRPO: group of responses scored together; within-group normalised reward replaces value network.

State Transition Table

Property	**Before (PPO)**	**After (GRPO)**
Critic / value network	Required	Not needed
Advantage estimation	GAE (per token)	Group normalisation
Memory overhead	2× model (actor + critic)	$\approx 1\times$ model
Samples per prompt	1	G (group sampling)
Variance reduction	Value baseline	Mean group reward

Table 13: State properties comparing PPO and GRPO.

Rejection Fine-Tuning (RFT)

Mathematical Foundation

RFT generates K candidate solutions per problem using the current policy, filters by a correctness verifier $\mathcal{V}$, and fine-tunes on the correct subset essentially a self-improvement loop without RL:

$$\mathcal{D}_{\text{correct}} = \{(x, y^{(k)}) : \mathcal{V}(x, y^{(k)}) = 1,\ y^{(k)} \sim \pi_\theta,\ k = 1..K\} \tag{31}$$

$$\mathcal{L}_{\text{RFT}} = -\sum_{(x,y)\in\mathcal{D}_{\text{correct}}} \sum_{t} \log P_\theta(y_t \mid x, y_{<t}) \tag{32}$$

The process repeats: train on correct outputs, generate new candidates, filter again.

Architecture Flow Diagram

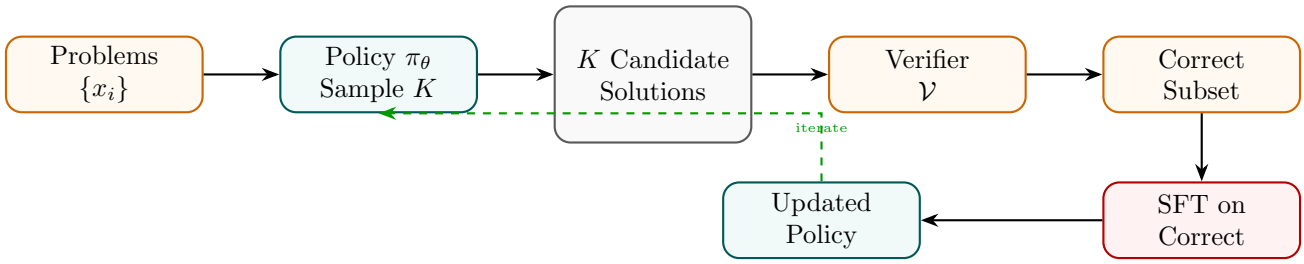

Figure 14: RFT: policy samples K solutions, verifier filters correct ones, SFT improves the policy iteratively.

State Transition Table

Property	Before	After
Data source	Static human labels	Self-generated correct solutions
RL component	Required (PPO)	None (SFT only)
Verifier	None	Required (exact-match, unit tests)
Pass@K improvement	Baseline	Improves with iterations
Training complexity	High (RL)	Standard SFT loop

Table 14: State properties before and after RFT.

REINFORCE

Mathematical Foundation

REINFORCE is the foundational policy gradient algorithm. For a trajectory $\tau = (s_0, a_0, s_1, a_1, \ldots)$, the gradient of expected return is:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[G(\tau) \nabla_\theta \log \pi_\theta(\tau)], \quad G(\tau) = \sum_{t=0}^{T} \gamma^t r_t \tag{33}$$

For LLMs, the trajectory is the token sequence, each token is an action, and r_T (end reward) is discounted back. A baseline b (e.g., mean reward) reduces variance:

$$\nabla_\theta J \approx \frac{1}{N} \sum_n (G_n - b) \nabla_\theta \log \pi_\theta(y_n | x_n) \tag{34}$$

Architecture Flow Diagram

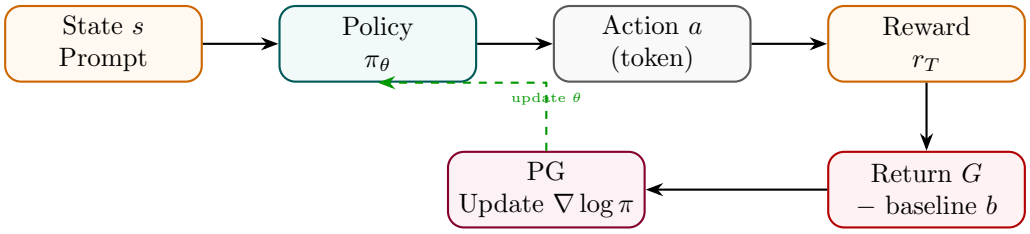

Figure 15: REINFORCE: policy samples action sequence, receives return G, updates via policy gradient.

State Transition Table

Property	Before	After
Gradient type	Supervised CE	Policy gradient $G\nabla \log \pi$
Variance	N/A	High (needs baseline)
Critic network	Not needed	Not needed
Reward type	N/A	Any differentiable or sparse
Update frequency	Per batch	Per episode/trajectory

Table 15: State properties before and after REINFORCE setup.

Advantage Actor-Critic (A2C / A3C)

Mathematical Foundation

A2C uses a shared backbone with two heads: a policy head (actor) and a value head (critic). The advantage $A(s,a) = Q(s,a) - V(s)$ reduces gradient variance:

$$\mathcal{L}_{\text{actor}} = -\mathbb{E}_t\big[A(s_t,a_t)\log \pi_\theta(a_t|s_t)\big] \tag{35}$$

$$\mathcal{L}_{\text{critic}} = \mathbb{E}_t\big[(r_t + \gamma V(s_{t+1}) - V(s_t))^2\big] \tag{36}$$

A3C extends A2C with asynchronous parallel workers each running independent environments and gradients are asynchronously applied to a central parameter server.

Architecture Flow Diagram

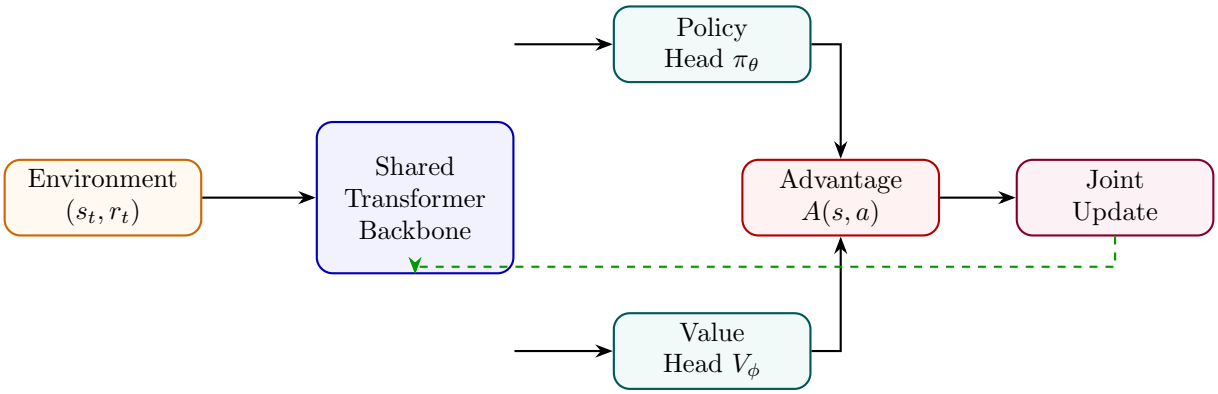

Figure 16: A2C: shared backbone feeds actor (policy) and critic (value) heads; advantage combines their outputs.

State Transition Table

Property	Before (REINFORCE)	After (A2C)
Variance reduction	Baseline only	Full advantage estimation
Value network	Separate	Shared backbone, separate head
Gradient stability	Low	Higher (advantage)
Parallelism	None	A3C: asynchronous workers
Sample efficiency	Low	Higher (on-policy)

Table 16: State properties comparing REINFORCE and A2C.

Self-Play Reinforcement Learning

Mathematical Foundation

In Self-Play RL, the model competes against previous versions of itself. At iteration n, the opponent is a snapshot $\pi_{\theta_{n-1}}$ (or mixture of past policies). The reward is determined by competitive outcome:

$$r(y_\theta, y_{\theta_{\text{old}}}) = \begin{cases} +1 & \text{if } y_\theta \text{ is judged better} \\ -1 & \text{otherwise} \end{cases} \tag{37}$$

$$\mathcal{J} = \mathbb{E}_{\pi_\theta, \pi_{\theta_{\text{old}}}} \left[r(y_\theta, y_{\theta_{\text{old}}}) \right] \tag{38}$$

This creates an ever-escalating difficulty curriculum since the opponent improves alongside the model.

Architecture Flow Diagram

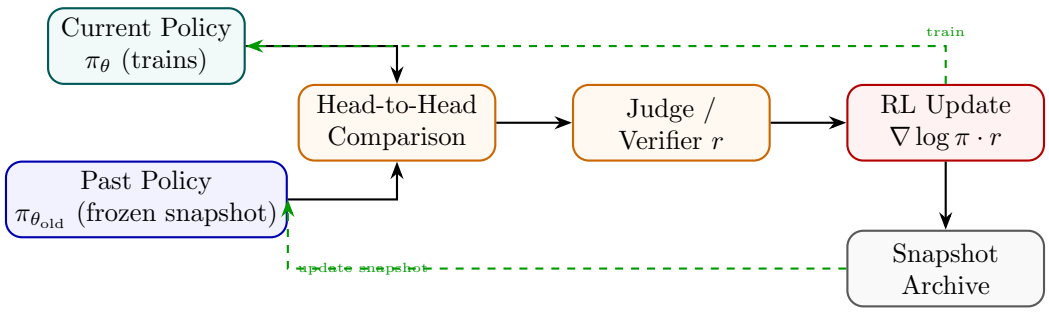

Figure 17: Self-Play RL: current policy competes against a frozen past snapshot; win/loss drives RL updates.

State Transition Table

Property	**Before**	**After**
Opponent	None / static	Evolving past self
Difficulty curriculum	Fixed	Auto-scaling
Reward source	External judge	Comparative win/loss
Policy snapshot archive	N/A	Maintained
Capability ceiling	Fixed by data	Self-determined

Table 17: State properties before and after Self-Play RL setup.

Monte Carlo Tree Search RL (MCTS-RL)

Mathematical Foundation

MCTS-RL builds a reasoning tree by alternating between *selection* (UCT formula), *expansion*, *rollout*, and *backpropagation*. The UCT score for node n:

$$\mathrm{UCT}(n) = \frac{W(n)}{N(n)} + c\sqrt{\frac{\ln N(\mathrm{parent}(n))}{N(n)}} \tag{39}$$

where $W(n)$ is total rollout reward and $N(n)$ is visit count. The policy is improved by training on trajectories weighted by MCTS visit counts:

$$\pi_\theta \leftarrow \pi_\theta + \eta\,\mathbb{E}\big[\text{MCTS visits}(a|s)\cdot\nabla_\theta \log \pi_\theta(a|s)\big] \tag{40}$$

Architecture Flow Diagram

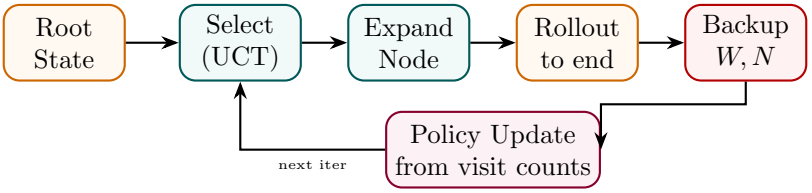

Figure 18: MCTS-RL: select (UCT), expand, rollout, backpropagate; visit counts guide policy improvement.

State Transition Table

Property	Before	After
Search strategy	Greedy / beam	Tree search (UCT)
State value estimate	None	$W(n)/N(n)$ from rollouts
Planning horizon	Local	Multi-step look-ahead
Compute per query	Inference only	Inference + tree expansion
Policy quality	Single-pass	Iteratively refined

Table 18: State properties before and after MCTS-RL integration.

Search-Augmented RL (Search-RL)

Mathematical Foundation

Search-RL augments the reasoning process with an explicit search over candidate next steps. At each step t, the policy generates B candidates $\{s_t^{(1)}, \ldots, s_t^{(B)}\}$, a verifier scores them, and the best is selected:

$$s_t^* = \arg\max_b V(x, s_{1:t-1}, s_t^{(b)}) \tag{41}$$

Training uses the full selected trajectory as RL experience:

$$\mathcal{L}_{\text{SearchRL}} = -\sum_t \mathbb{1}[\mathcal{V}(x, y) = 1] \log \pi_\theta(s_t^* \mid x, s_{1:t-1}) \tag{42}$$

Architecture Flow Diagram

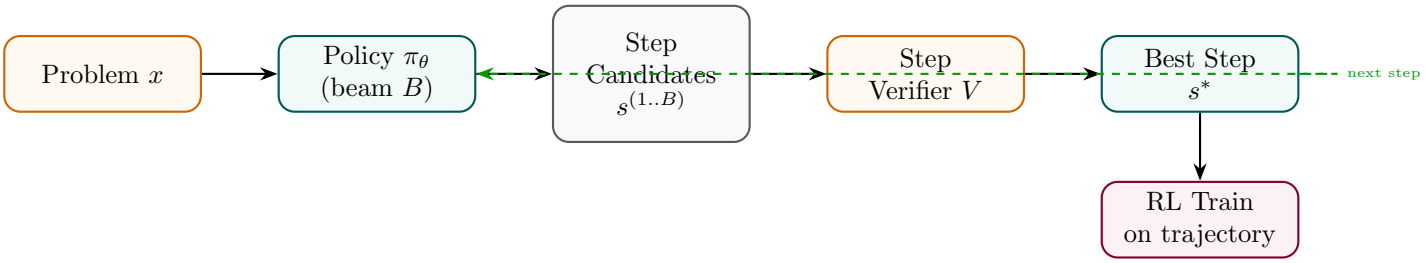

Figure 19: Search-RL: at each step, beam search generates candidates; verifier selects best; policy trained on trajectory.

State Transition Table

Property	Before	After
Step selection	Greedy / top-1	Verifier-guided beam search
Search breadth	1	B candidates per step
Verifier	None	Step-level verifier V
Training signal	Outcome reward	Step-selected trajectory
Error recovery	None	Search can backtrack

Table 19: State properties before and after Search-RL.

Tree-of-Thought Training

Mathematical Foundation

Tree-of-Thought (ToT) Training teaches the model to generate, evaluate, and prune multiple reasoning branches. The model is trained on annotated trees where each branch b_i has a quality score $q_i \in [0, 1]$. The training objective rewards correct branches and penalises dead-ends:

$$\mathcal{L}_{\text{ToT}} = -\sum_i q_i \sum_t \log P_\theta(b_{i,t} \mid x, b_{i,<t}) + (1 - q_i) \sum_t \log(1 - P_\theta(b_{i,t} \mid x, b_{i,<t})) \tag{43}$$

A separate scoring model $s_\phi(x, b_i)$ is trained to evaluate intermediate thought nodes and guide beam search at inference.

Architecture Flow Diagram

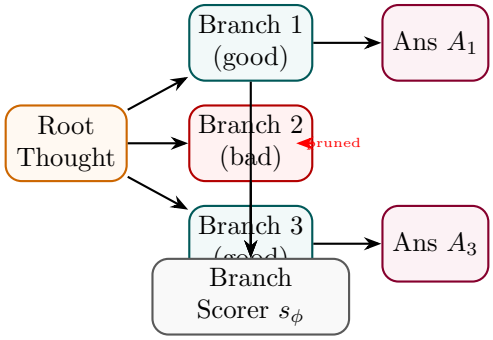

Figure 20: ToT Training: good branches lead to answers; bad branches are pruned; scorer trained to evaluate all.

State Transition Table

Property	Before	After
Reasoning structure	Linear chain	Branching tree
Dead-end handling	None	Explicit pruning
Branch evaluator	None	Trained s_ϕ
Search at inference	Linear	Guided tree beam
Complex planning	Weak	Strong

Table 20: State properties before and after Tree-of-Thought Training.

Self-Consistency Training

Mathematical Foundation

Self-Consistency Training generates K independent reasoning paths and uses majority voting to determine the correct answer. The model is then trained to increase probability of the majority-vote answer y^*:

$$y^* = \arg\max_y \sum_{k=1}^{K} \mathbf{1}[\text{answer}(y^{(k)}) = y], \quad y^{(k)} \sim \pi_\theta(\cdot|x) \tag{44}$$

$$\mathcal{L}_{\text{SC}} = -\sum_{\{k:\,\text{answer}(y^{(k)})=y^*\}} \sum_t \log P_\theta(y_t^{(k)} \mid x, y_{<t}^{(k)}) \tag{45}$$

Paths that agree with the majority vote are reinforced; minority paths are not trained on (or penalised).

Architecture Flow Diagram

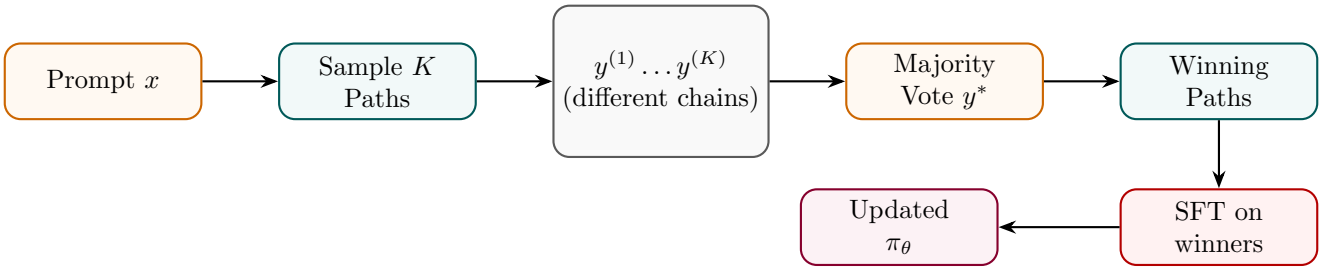

Figure 21: Self-Consistency Training: majority-vote over K paths identifies winning chains; model trained on those.

State Transition Table

Property	Before	After
Inference mode	Single greedy	K diverse samples
Answer selection	Argmax	Majority vote
Training target	All completions	Majority-consistent paths only
Accuracy ceiling	Limited by single pass	Higher (ensemble effect)
Compute cost	$1\times$	$K\times$ at sampling time

Table 21: State properties before and after Self-Consistency Training.

Part III
Reward Modeling

Reward Model (RM)

Mathematical Foundation

A Reward Model takes a prompt x and a completion y and outputs a scalar preference score. It is trained on human-ranked pairs $(y_w \succ y_l)$ via the Bradley-Terry log-likelihood:

$$\mathcal{L}_{\text{RM}} = -\mathbb{E}_{(x,y_w,y_l)}\left[\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))\right] \tag{46}$$

The RM is typically initialised from a fine-tuned LLM with the final token's hidden state projected to a scalar via a linear head $W_r \in \mathbb{R}^{d\times 1}$:

$$r_\phi(x, y) = W_r^\top h_{\text{last}}(x, y) \tag{47}$$

Architecture Flow Diagram

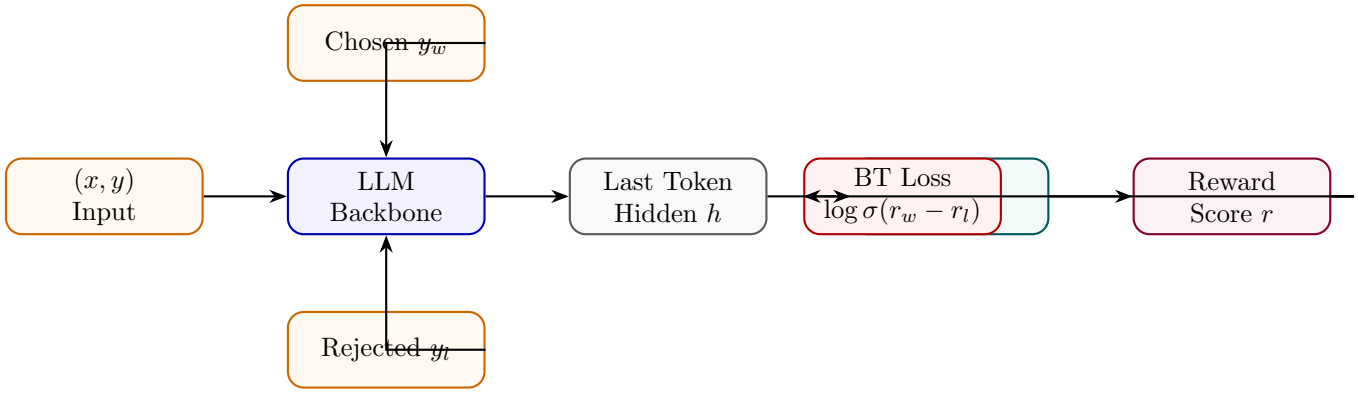

Figure 22: Reward Model: LLM backbone maps (x, y) pair to scalar via linear head; trained on chosen/rejected pairs.

State Transition Table

Property	Before	After
Output type	Token distribution	Scalar reward
Final layer	LM head	Scalar head W_r
Training data	Text completions	(x, y_w, y_l) preference pairs
Loss function	Cross-entropy	Bradley-Terry log-likelihood
Usage	Language generation	RLHF reward signal

Table 22: State properties before and after reward model training.

Process Reward Model (PRM)

Mathematical Foundation

A PRM assigns a reward to each intermediate reasoning step s_t rather than to the final answer alone. Given a solution prefix $(s_1, \ldots, s_t)$, the PRM outputs a binary step quality $q_t \in \{0, 1\}$:

$$r_{\text{PRM}}(x, s_{1:T}) = \prod_{t=1}^{T} r_t, \quad r_t = P_\phi(q_t = 1 \mid x, s_{1:t}) \tag{48}$$

The PRM is trained on step-level human labels with binary cross-entropy:

$$\mathcal{L}_{\text{PRM}} = -\sum_t \big[q_t \log r_t + (1 - q_t) \log(1 - r_t)\big] \tag{49}$$

Architecture Flow Diagram

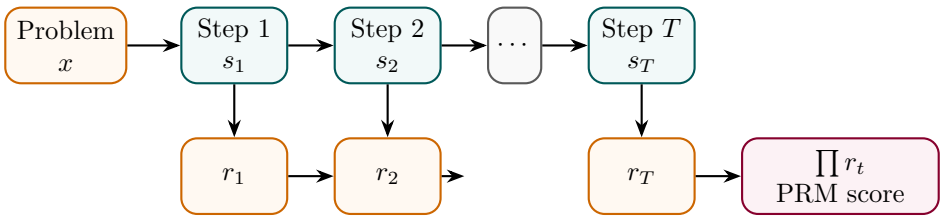

Figure 23: PRM: each reasoning step independently scored; product of step scores gives overall process reward.

State Transition Table

Property	**Before (RM)**	**After (PRM)**
Reward granularity	Final answer only	Per-step
Error localisation	None	Identifies failing step
Label annotation	Answer-level	Step-level (expensive)
Reward signal density	Sparse (1 per traj.)	Dense (T per traj.)
Training on correct solutions	Required	Not required

Table 23: State properties comparing RM and PRM.

Outcome Reward Model (ORM)

Mathematical Foundation

An ORM judges only the *correctness of the final answer* y_{ans}, ignoring intermediate steps. For verifiable tasks (math, code), correctness can be checked automatically:

$$r_{\text{ORM}}(x, y) = \mathbf{1}[\text{verify}(y_{\text{ans}}, y^*_{\text{gold}})] \tag{50}$$

For open-ended tasks, the ORM is a learned binary classifier trained on (correct, incorrect) outcome labels:

$$\mathcal{L}_{\text{ORM}} = -\big[c \log P_\phi(y) + (1 - c) \log(1 - P_\phi(y))\big], \quad c \in \{0, 1\} \tag{51}$$

Architecture Flow Diagram

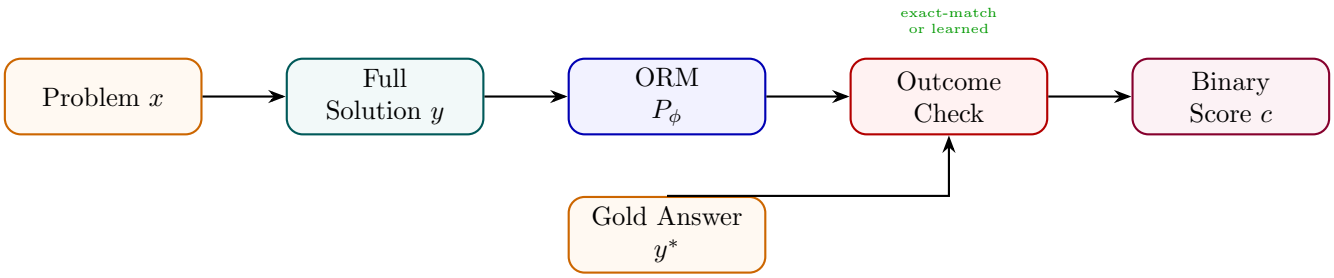

Figure 24: ORM: scores the final answer against gold; can be rule-based (exact-match) or learned classifier.

State Transition Table

Property	Before	After
Reward scope	N/A	Final answer only
Intermediate credit	None	None
Annotation cost	N/A	Low (binary label)
Verifiable tasks	Manual check	Automatic (rule-based)
Open-ended tasks	None	Learned classifier

Table 24: State properties before and after ORM setup.

Verifier Model

Mathematical Foundation

A Verifier Model is a binary classifier that determines whether a candidate solution is correct. Unlike an ORM that outputs a probability, verifiers typically use a hard decision rule learned from (correct, incorrect) solution pairs:

$$\hat{v}(x, y) = \mathbf{1}\big[P_\phi(\text{correct} \mid x, y) \geq \tau\big] \tag{52}$$

Verifiers are used in test-time search (Best-of-N) and training-time filtering (RFT). The decision threshold τ trades off precision and recall on the verification task:

$$\mathcal{L}_{\text{ver}} = -\big[v \log P_\phi + (1 - v) \log(1 - P_\phi)\big], \quad v \in \{0, 1\} \tag{53}$$

Architecture Flow Diagram

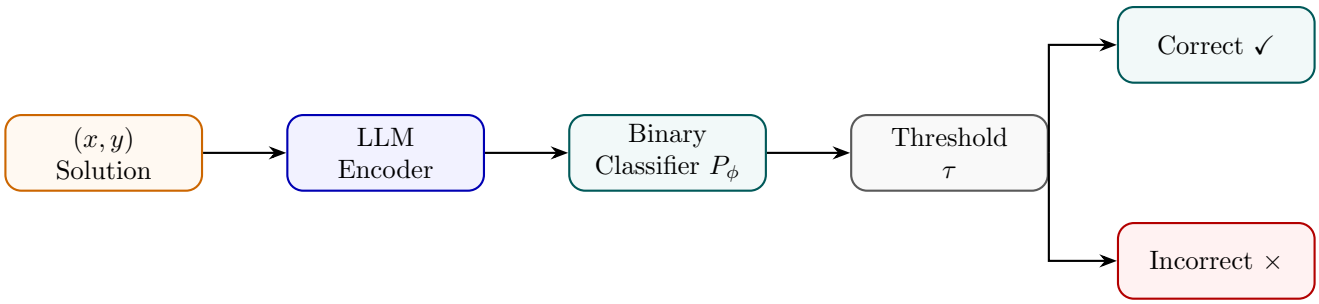

Figure 25: Verifier Model: encodes solution, classifies as correct/incorrect via learned threshold.

State Transition Table

Property	Before	After
Output	Continuous score	Binary decision
Threshold	N/A	Tunable τ
Primary use	Ranking	Filtering (accept/reject)
Training data	Ranked pairs	Correct/incorrect labels
Precision/recall trade-off	N/A	Controlled via τ

Table 25: State properties before and after Verifier Model training.

Critic Model

Mathematical Foundation

A Critic Model generates natural-language *critiques* of candidate outputs and optionally a numerical score. It is trained on (output, critique, revision) triplets. The critique generation loss:

$$\mathcal{L}_{\text{crit}} = -\sum_{t} \log P_\phi(c_t \mid x, y, c_{<t}) \tag{54}$$

where c is the gold critique. Critics enable self-refinement loops: the model generates y, the critic evaluates it producing c, and the model revises $y \to y'$ conditioned on (x, y, c):

$$y' \sim P_\theta(\cdot \mid x, y, c) \tag{55}$$

Architecture Flow Diagram

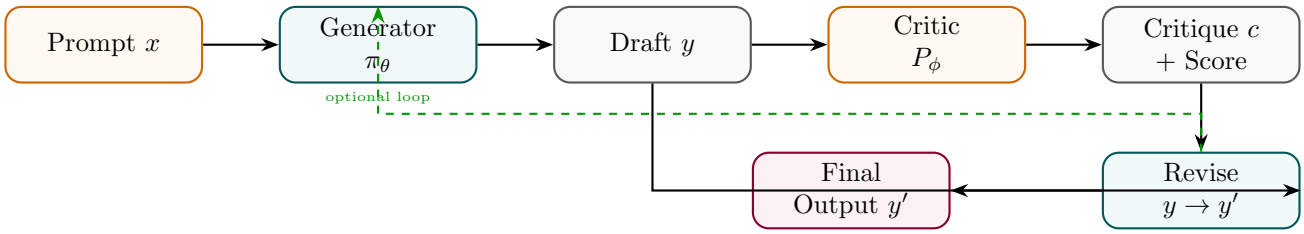

Figure 26: Critic Model: generates natural-language critique of a draft; generator revises using the critique.

State Transition Table

Property	Before	After
Feedback type	Scalar reward	Natural-language critique
Interpretability	Low (scalar)	High (textual feedback)
Revision loop	Not applicable	Generator refines on critique
Training data	Preference pairs	(output, critique, revision) triplets
Use case	RM scoring	Self-refinement / RLHF

Table 26: State properties before and after Critic Model training.